

~\$ compendio-gentoo --install

Resumen explicado, paso a paso, de todo el proceso de instalación de Gentoo Linux amd64: de medio de arranque a sistema encendido. Pensado para quien nunca ha instalado Gentoo: cada comando se explica en palabras simples, con ejemplos y tabla de opciones.

init: **OpenRC**

tabla: **GPT + UEFI** (y alternativa MBR/BIOS)

arranque: **GRUB**

initramfs: **dracut**

perfil: **multilib**

📖 ANTES DE EMPEZAR – GLOSARIO MÍNIMO

Partición

Una división del disco físico. Un solo disco se puede "cortar" en varios pedazos independientes, cada uno con su propio contenido.

Punto de montaje

La carpeta a través de la cual accedes al contenido de una partición. Por ejemplo, la partición raíz se monta en `/`.

MBR vs GPT

Son dos formas distintas de organizar la tabla de particiones en el disco. MBR es la antigua (asociada a BIOS); GPT es la moderna (asociada a UEFI, y recomendada siempre que se pueda).

Sistema de archivos (filesystem)

La forma en que se organizan los archivos dentro de una partición (ej. FAT32, ext4, xfs). Sin esto, la partición es solo espacio vacío sin estructura.

Firmware BIOS vs UEFI

El programa que arranca la computadora antes del sistema operativo. BIOS es el estándar antiguo (equipos pre-2010); UEFI es el moderno, más flexible y con más funciones.

Terminal / shell

La ventana de texto donde escribes comandos. En este documento cada línea que empieza con `root #` se escribe ahí y se confirma con Enter.

ÍNDICE

1. Particionado del disco (fdisk y cfdisk, EFI y BIOS)
2. Sistemas de archivos y swap
3. Stage3: montaje y extracción
4. Chroot: entrar al nuevo sistema
5. Portage: sync, perfil y USE flags
6. Kernel, initramfs y firmware
7. Sistema base: locale, zona horaria, red
8. fstab
9. Bootloader: GRUB
10. Últimos pasos y primer arranque
11. Bifurcación: instalación mínima vs gráfica
12. Comandos esenciales post-instalación

Este es el primer paso "real" sobre el disco duro. Antes de instalar nada hay que decidir en cuántos pedazos (particiones) se va a dividir, y de qué tamaño. Aquí se muestran **dos herramientas** (fdisk, en texto por comandos, y cfdisk, visual con flechas) y **dos esquemas** según el tipo de firmware de la máquina (UEFI o BIOS).

¿Cómo sé si mi equipo usa BIOS o UEFI? Casi cualquier equipo fabricado después de 2010 usa UEFI. Si el medio de instalación arrancó mostrando un menú gráfico o detecta `/sys/firmware/efi`, está en modo UEFI. Ante la duda, usa el esquema UEFI + GPT: es el recomendado por el propio Handbook de Gentoo.

/dev/sda — disco GPT (UEFI)

sda1 · /efi · FAT32 · ESP · ~1 GiB

sda2 · swap · linux-swap · ≈2×RAM

sda3 · / · xfs/ext4 · resto del disco

Esquema UEFI + GPT (recomendado)

/dev/sda — disco MBR (BIOS/legacy)

sda1 · /boot · xfs/ext4 · ~500 MiB–1 GiB

sda2 · swap · linux-swap · ≈2×RAM

sda3 · / · xfs/ext4 · resto del disco

Esquema MBR + BIOS/legacy (equipos antiguos)

La diferencia clave entre ambos esquemas: en **UEFI** la primera partición es la **ESP** (EFI System Partition), formateada en FAT32, donde el firmware busca el cargador de arranque (GRUB). En **BIOS/legacy** no existe ESP; en su lugar hay una partición `/boot` normal donde GRUB coloca el kernel, y el propio código de arranque de GRUB se escribe directamente al inicio del disco (fuera de cualquier partición).

Opción A — fdisk (por comandos de una letra)

fdisk es la herramienta clásica: se abre en modo interactivo y cada acción se activa escribiendo una sola letra seguida de Enter. No es gráfica, pero es muy precisa y está disponible en cualquier medio de instalación.

📁 fdisk — abrir el editor de particiones

BAJO RIESGO ▼

US0

```
root # fdisk /dev/sda
```

Para qué sirve: abre el disco indicado en un modo de edición donde puedes ver, crear, borrar y modificar particiones. Mientras no escribas la letra `w` (write/guardar), **nada se modifica de verdad en el disco**: todo son cambios "en memoria" que se pueden abortar con `q`.

Ejemplo — así se ve al entrar

```
Command (m for help): p
Disk /dev/sda: 931.51 GiB, 1000204886016 bytes
Disklabel type: gpt
Device      Start      End      Sectors  Size Type
/dev/sda1   2048     2099199   2097152    1G EFI System
```

TECLA	QUÉ HACE, EXPLICADO SIMPLE
-------	----------------------------

m	Muestra la ayuda con todas las letras disponibles. Úsala si te pierdes.
---	---

p	"Print": imprime en pantalla cómo quedaría la tabla de particiones ahora mismo.
---	---

g	Crea una tabla de particiones GPT nueva y vacía (borra en memoria lo que había).
---	---

o	Crea una tabla de particiones MBR/DOS nueva y vacía (para el esquema BIOS/legacy).
---	---

n	"New": crea una partición nueva. Preguntará número, sector de inicio y tamaño.
---	--

t	"Type": cambia el tipo de una partición ya creada (ej. marcarla como "EFI System").
---	---

d	"Delete": borra una partición (solo en memoria, hasta que uses w).
---	--

w	"Write": guarda todos los cambios de verdad en el disco. Sin retorno.
---	--

q	"Quit": sale sin guardar nada. Útil si te equivocaste.
---	--

Afecta: la tabla de particiones del disco completo. Solo se hace permanente al presionar w; los datos de particiones borradas se pierden en ese momento.

fdisk – crear el esquema UEFI + GPT paso a paso

RIESGO MEDIO ▼

UEFI · GPT

Secuencia completa dentro de `fdisk /dev/sda`, creando las tres particiones del esquema recomendado: ESP, swap y raíz.

1. CREAR LA TABLA GPT (BORRA PARTICIONES PREVIAS)

```
Command (m for help): g
Created a new GPT disklabel.
```

2. CREAR LA ESP (PARTICIÓN 1, 1 GIB, TIPO EFI SYSTEM)

```
Command (m for help): n
Partition number (1-128, default 1): 1
First sector: [Enter]           # usa el valor por defecto
Last sector: +1G               # tamaño: 1 gibibyte

Command (m for help): t
Selected partition 1
Type: 1                        # "EFI System"
```

3. CREAR LA PARTICIÓN SWAP (PARTICIÓN 2, 4 GIB DE EJEMPLO)

```
Command (m for help): n
Partition number (2-128, default 2): 2
First sector: [Enter]
Last sector: +4G

Command (m for help): t
```

```
Partition number: 2
Type: 19                                     # "Linux swap"
```

4. CREAR LA PARTICIÓN RAÍZ CON EL RESTO DEL DISCO

```
Command (m for help): n
Partition number (3-128, default 3): 3
First sector: [Enter]
Last sector: [Enter]                        # usa todo el espacio restante
```

5. REVISAR Y GUARDAR

```
Command (m for help): p                    # revisa que todo esté correcto
Command (m for help): w                    # escribe los cambios al disco
```

VALOR ESCRITO	QUÉ SIGNIFICA
+1G / +4G	Tamaño relativo al sector de inicio: "+1 Gibibyte a partir de aquí". También acepta M (mebibytes) o T (tebibytes).
[Enter] en "Last sector"	Acepta el valor por defecto que fdisk sugiere; en la última partición eso significa "usa todo el espacio libre restante".
Type 1 / 19	Código numérico corto del tipo de partición en la lista que fdisk muestra al usar t; no hace falta memorizarlos, fdisk permite buscarlos escribiendo L para listarlos.

Afecta: crea las tres particiones que usará todo el resto de la instalación. Nada se pierde hasta el paso 5 (w).

fdisk – crear el esquema MBR + BIOS/legacy paso a paso

RIESGO MEDIO ▼

BIOS · MBR

Mismo proceso, pero con tabla MBR en vez de GPT y sin ESP: la primera partición ahora es un /boot normal, no FAT32 obligatorio.

1. CREAR LA TABLA MBR/DOS

```
Command (m for help): o
Created a new DOS disklabel.
```

2. PARTICIÓN DE ARRANQUE (/BOOT, ~1 GIB)

```
Command (m for help): n
Partition type: p                        # primaria
Partition number (1-4, default 1): 1
First sector: [Enter]
Last sector: +1G
```

3. PARTICIÓN SWAP

```
Command (m for help): n
Partition type: p
Partition number (2-4, default 2): 2
```

```
First sector: [Enter]
Last sector: +4G

Command (m for help): t
Partition number: 2
Hex code: 82                # "Linux swap / Solaris"
```

4. PARTICIÓN RAÍZ (RESTO DEL DISCO) Y GUARDAR

```
Command (m for help): n
Partition type: p
Partition number (3-4, default 3): 3
First sector: [Enter]
Last sector: [Enter]

Command (m for help): w
```

Nota: en discos MBR sólo se permiten 4 particiones "primarias". Si en algún momento necesitas más de 4, una de ellas debe convertirse en "extendida" para alojar particiones "lógicas" adicionales; para un esquema de 3 particiones como este no hace falta.

Afecta: crea el esquema clásico de 3 particiones para arranque en modo BIOS/legacy.

Opción B – cfdisk (menú visual, más amigable)

cfdisk hace exactamente lo mismo que fdisk, pero con una interfaz de pantalla completa: se navega con las flechas del teclado y las acciones aparecen como un menú abajo. Es la opción más recomendable si es la primera vez que particionas un disco, porque es más difícil perderse.

cfdisk – particionar con menús y flechas

BAJO RIESGO

USO

```
root # cfdisk /dev/sda
```

Para qué sirve: es una versión visual de fdisk. Muestra la lista de particiones en pantalla completa y, abajo, un menú de acciones (New, Delete, Type, Write, Quit) que se elige con las flechas ← → y se confirma con Enter. Ideal para ver de un vistazo cuánto espacio libre queda mientras divides el disco.

TECLA

QUÉ HACE, EXPLICADO SIMPLE

↑ ↓

Mueve la selección entre las particiones existentes (o el espacio libre).

← →

Mueve la selección entre las opciones del menú inferior (New, Delete, etc.).

Enter

Confirma la opción de menú resaltada.

[New]

Crea una partición nueva en el espacio libre seleccionado; pide el tamaño (ej. escribir 1G y Enter).

[Delete]

Borra la partición seleccionada (solo en memoria, como en fdisk).

[Type]

Abre una lista con nombres de tipo legibles (ej. "EFI System", "Linux swap") para asignar a la partición seleccionada — más fácil que recordar códigos.

[Bootable]

Marca la partición como arrancable (solo relevante en esquemas MBR/BIOS).

[Write]

Guarda los cambios de verdad en el disco; pide escribir la palabra **yes** para confirmar.

[Quit]

Sale del programa sin guardar cambios pendientes.

Ejemplo de flujo típico (esquema UEFI)

1. Seleccionar el espacio libre → [New] → escribir 1G → Enter
2. Con la partición nueva resaltada → [Type] → elegir "EFI System"
3. Bajar al siguiente espacio libre → [New] → escribir 4G → [Type] → "Linux swap"
4. Último espacio libre → [New] → Enter (usa todo lo que queda)
5. [Write] → escribir "yes" → Enter → [Quit]

Afecta: exactamente lo mismo que fdisk — la tabla de particiones del disco. Solo cambia la forma de interactuar, el resultado final es idéntico.

mkswap / swapon — activar el intercambio

BAJO RIESGO

USO

```
root # mkswap /dev/sda2
root # swapon /dev/sda2
```

Para qué sirve: la partición swap es "memoria RAM de respaldo" en disco: cuando la RAM física se llena, el sistema mueve ahí datos poco usados. mkswap prepara la partición con la estructura necesaria (como formatearla, pero específico para swap). swapon la activa para que el sistema empiece a usarla de inmediato.

Este paso de activarla a mano solo es necesario en el medio en vivo. Una vez el sistema esté instalado y arrancando desde el disco, será /etc/fstab (ver parte 08) quien la active automáticamente en cada arranque.

Afecta: solo la partición indicada. No toca el resto del disco.

02

Sistemas de archivos y montaje

Con las particiones ya creadas, hay que darles un sistema de archivos (formatearlas) y ponerlas a disposición del instalador.

mkfs.vfat / mkfs.xfs — formatear particiones

RIESGO MEDIO

```
root # mkfs.vfat -F 32 /dev/sda1 # ESP
root # mkfs.xfs /dev/sda3 # raíz
```

```
root # mkfs.xfs /dev/sda1 # /boot
root # mkfs.xfs /dev/sda3 # raíz
```

Para qué sirve: "formatear" significa escribir la estructura de un sistema de archivos vacío sobre la partición, algo así como poner estanterías vacías dentro de una habitación recién construida. Sin este paso la partición existe pero no se puede guardar nada organizado ahí.

OPCIÓN	EFEECTO
--------	---------

-F 32	Fuerza la variante FAT32 (obligatoria para la ESP; sin esta opción mkfs.vfat podría elegir FAT16).
-------	--

La ESP **debe** ser FAT (FAT32) porque así lo exige la especificación UEFI: es el único formato que el firmware sabe leer antes de que exista un sistema operativo. La partición raíz puede usar xfs, ext4, btrfs, etc. — xfs es la que trae por defecto el Handbook de Gentoo en sus ejemplos.

Afecta: borra todo el contenido previo de esa partición. No hay confirmación adicional, así que hay que revisar dos veces el nombre del dispositivo (/dev/sda1, etc.) antes de ejecutar.

mkdir / mount – puntos de montaje

BAJO RIESGO

```
root # mkdir --parents /mnt/gentoo
root # mount /dev/sda3 /mnt/gentoo
root # mkdir --parents /mnt/gentoo/efi
```

Para qué sirve: mkdir crea la carpeta que hará de "puerta de entrada" a la partición. mount conecta la partición formateada con esa carpeta: a partir de ese momento, todo lo que se guarde dentro de /mnt/gentoo en realidad se está guardando en /dev/sda3.

OPCIÓN	EFEECTO
--------	---------

--parents	Crea también las carpetas intermedias que falten en la ruta, sin dar error si alguna ya existe.
-----------	---

Afecta: solo la vista de archivos del entorno en vivo actual; no modifica el contenido del disco.

El **stage3** es la semilla del sistema: un sistema Gentoo mínimo ya compilado que se descomprime sobre la raíz, como un molde base a partir del cual se construye todo lo demás.

↓ tar – extraer el stage3

NO INTERRUMPIR

USO

```
root # tar xpvf stage3-*.tar.xz --xattrs-include='*.*' \
--numeric-owner -C /mnt/gentoo
```

Para qué sirve: descomprime el archivo comprimido del stage3 directamente dentro de /mnt/gentoo, recreando ahí todos los archivos y carpetas de un sistema Gentoo mínimo (bibliotecas, herramientas básicas, Portage, etc.), preservando permisos y propietarios tal como los definió Gentoo.

OPCIÓN	EFEECTO
x	Extrae el contenido del archivo (de "extract").
p	Preserva los permisos originales de cada archivo (de "preserve").
v	Salida detallada: muestra en pantalla cada archivo mientras se extrae (de "verbose"). Se puede omitir si molesta el exceso de texto.
f	Indica cuál es el archivo de entrada (de "file"); siempre va seguido del nombre del tarball.
--xattrs-include='*.*'	Conserva atributos extendidos (metadatos especiales) en todos los espacios de nombres.
--numeric-owner	Usa los números de usuario/grupo del archivo tal cual, en vez de traducirlos por nombre, evitando mezclas con las cuentas del medio en vivo.
-C /mnt/gentoo	Cambia el directorio de trabajo a /mnt/gentoo antes de extraer (de "change directory"), así el contenido queda ahí sin importar desde dónde se ejecute el comando.

Afecta: puebla por completo /mnt/gentoo con el sistema base. Es un paso **inseguro de interrumpir**: un corte de luz o un Ctrl+C a mitad de la extracción puede dejar el sistema base incompleto y corrupto.

📄 nano /etc/portage/make.conf – flags de compilación

BAJO RIESGO

Para qué sirve: make.conf es el archivo de configuración principal de Portage (el gestor de paquetes de Gentoo). Aquí se definen, entre otras cosas, cómo debe compilarse cada programa que se instale. **Se lee en cada emerge**, no es un ajuste de una sola vez.

EJEMPLO DE CONTENIDO

```
COMMON_FLAGS="-march=native -O2 -pipe"
CFLAGS="${COMMON_FLAGS}"
CXXFLAGS="${COMMON_FLAGS}"
MAKEOPTS="-j5"
```


VARIABLE	EFEECTO
CFLAGS / CXXFLAGS	Optimizaciones que el compilador aplica al construir cada paquete desde el código fuente (por ejemplo, aprovechar instrucciones específicas del procesador).
MAKEOPTS="-j N"	Número de procesos de compilación en paralelo. Un valor común es núcleos de CPU + 1. Portage detecta hilos de CPU, pero no la RAM disponible, así que en equipos con poca memoria conviene bajar este número a mano.

Afecta: el rendimiento y el tiempo de compilación de todo paquete construido desde ese momento en adelante, incluyendo el propio sistema.

04

Chroot: entrar al nuevo sistema

Se «cambia de raíz» para que todos los comandos siguientes actúen dentro del sistema recién instalado, no en el medio en vivo. Es como abrir una puerta hacia el sistema nuevo sin tener que reiniciar la computadora.

medio en vivo – / (raíz temporal)

```
/proc /sys /dev /run
se montan (bind)
dentro de /mnt/gentoo
/etc/resolv.conf
se copia (cp --dereference)
```



```
chroot /mnt/gentoo /bin/bash
la raíz "/" pasa a ser /mnt/gentoo
source /etc/profile
export PS1="(chroot) ${PS1}"
recarga variables y marca el prompt
```

Secuencia de entrada al chroot

mount --rbind / chroot – entrar al entorno nuevo

PUNTO DE NO INTERRUPCIÓN ▾

PREPARAR FILESYSTEMS VIRTUALES

```
root # cp --dereference /etc/resolv.conf /mnt/gentoo/etc/
root # mount --types proc /proc /mnt/gentoo/proc
root # mount --rbind /sys /mnt/gentoo/sys
root # mount --make-rslave /mnt/gentoo/sys
root # mount --rbind /dev /mnt/gentoo/dev
root # mount --make-rslave /mnt/gentoo/dev
root # mount --bind /run /mnt/gentoo/run
root # mount --make-slave /mnt/gentoo/run
```

ENTRAR

```
root # chroot /mnt/gentoo /bin/bash
root # source /etc/profile
root # export PS1="(chroot) ${PS1}"
```

Para qué sirve todo esto: el sistema nuevo necesita "ver" ciertas carpetas especiales del kernel (/proc, /sys, /dev, /run) para funcionar correctamente incluso estando dentro del chroot. Estos comandos las conectan (montan) dentro de /mnt/gentoo antes de "entrar". Copiar resolv.conf asegura que, dentro del sistema nuevo, siga funcionando la resolución de nombres de dominio (para poder seguir descargando cosas de internet).

COMANDO/OPCIÓN	EFEECTO
<code>cp --dereference</code>	Copia el archivo real al que apunta un enlace simbólico, no el enlace en sí, para que la configuración de DNS funcione dentro del chroot.
<code>--rbind</code>	Vuelve a montar recursivamente un árbol de carpetas en otra ruta: crea un segundo punto de entrada al mismo contenido, no una copia.
<code>--make-rslave</code>	Evita que los cambios de montaje hechos dentro del chroot se propaguen de vuelta al sistema en vivo.
<code>chroot <ruta> /bin/bash</code>	Cambia la carpeta raíz "/" de la sesión actual a <ruta> y abre ahí dentro una terminal (bash) nueva.
<code>source /etc/profile</code>	Vuelve a cargar variables de entorno (como la ruta PATH) ya estando dentro del nuevo sistema.
<code>export PS1="(chroot) \${PS1}"</code>	Cambia el texto del prompt para recordar visualmente que se está dentro del chroot y no en el medio en vivo.

Si se usa el medio de instalación oficial de Gentoo, todo este bloque se puede sustituir por un solo comando: `arch-chroot /mnt/gentoo`, que automatiza los montajes.

Afecta: a partir de aquí, cada comando actúa sobre el sistema instalado, no sobre el medio en vivo. Si la instalación se interrumpe después de este punto, se puede **reanudar** repitiendo este bloque tras volver a montar las particiones.

05

Portage: sync, perfil y USE flags

Portage es el gestor de paquetes de Gentoo. Antes de instalar software hay que sincronizar el árbol de ebuilds (el catálogo) y elegir un perfil.

 `emerge-webrsync / eselect profile – árbol y perfil`

BAJO RIESGO

USO

```
root # emerge-webrsync
root # eselect profile list
root # eselect profile set 1
```

Para qué sirve: `emerge-webrsync` descarga una instantánea del árbol de ebuilds: el catálogo completo de todos los paquetes que Gentoo sabe instalar, junto con sus recetas de compilación. Sin este paso, Portage no sabría que existen esos paquetes. `eselect profile muestra (list)` y

selecciona (set) el **perfil**: la plantilla base de USE flags, versión de compilador y opciones de sistema (ej. desktop, servidor, multilib).

Afecta: qué paquetes existen para instalar y qué opciones traen activadas por defecto.

USE flags – activar/desactivar características

BAJO RIESGO

Para qué sirve: las USE flags funcionan como interruptores que dicen "compila este paquete con soporte para X" o "sin soporte para X". Por ejemplo, la flag X activa soporte gráfico, y pulseaudio activa integración de audio. **Nunca están "ausentes"**: cada paquete usa su valor por defecto (heredado del perfil) salvo que se indique lo contrario.

EJEMPLO – GLOBAL (TODO EL SISTEMA)

```
# /etc/portage/make.conf
USE="X wayland pulseaudio"
VIDEO_CARDS="amdgpu radeonsi"
CPU_FLAGS_X86="mmx sse sse2 avx"
```

EJEMPLO – SOLO PARA UN PAQUETE CONCRETO

```
# /etc/portage/package.use/media
media-video/ffmpeg x264 x265
```

La segunda forma existe para no ensuciar la configuración global: en vez de activar una flag para *todos* los paquetes, se activa solo para uno en concreto, dentro de la carpeta `/etc/portage/package.use/` (se pueden crear tantos archivos ahí como se quiera, con cualquier nombre).

VARIABLE	EFEECTO
USE	Lista global de características a compilar/omitir en todos los paquetes.
VIDEO_CARDS	Controladores gráficos a compilar. Es independiente de CPU_FLAGS_X86: una controla la GPU, la otra el procesador.
CPU_FLAGS_X86	Extensiones de instrucciones del procesador a aprovechar en paquetes compatibles (se puede detectar automáticamente con la herramienta <code>cpuid2cpuflags</code>).

Afecta: qué funcionalidades trae cada paquete al compilarse; cambiarlas después obliga a recompilar con `emerge -N @world`.

emerge – instalar y actualizar paquetes

BAJO RIESGO

US0

```
root # emerge --ask app-editors/vim
root # emerge --ask --update --deep --newuse @world
```

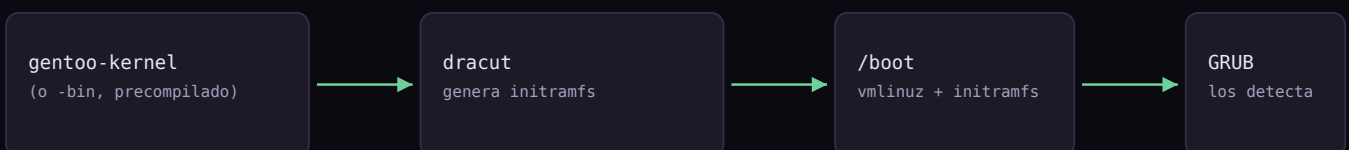
Para qué sirve: emerge es el comando que instala software en Gentoo. A diferencia de otras distribuciones, normalmente compila el programa desde su código fuente en ese momento (aunque también existen paquetes binarios precompilados).

OPCIÓN	EFFECTO
<code>--ask</code>	Muestra el plan de instalación (qué se va a instalar y qué tamaño tiene) y pide confirmación (s/n) antes de empezar a compilar.
<code>--update</code>	Actualiza paquetes ya instalados a versiones más nuevas disponibles.
<code>--deep</code>	Revisa también las dependencias de las dependencias, no solo las de primer nivel.
<code>--newuse</code>	Recompila paquetes cuyas USE flags relevantes cambiaron desde la última vez que se instalaron.
<code>@world</code>	Un "conjunto" especial: todos los paquetes instalados explícitamente por el usuario (no las dependencias automáticas).

Afecta: instala, actualiza o recompila software; puede tardar desde segundos hasta horas según el paquete y la potencia del equipo.

06 Kernel, initramfs y firmware

El kernel es el núcleo del sistema operativo. Puede instalarse ya compilado (kernel de distribución) o compilarse a mano. Con dracut se genera el initramfs necesario para arrancar.



Del paquete del kernel al menú de arranque



Kernel de distribución (recomendado para empezar)

BAJO RIESGO

USO

```
# /etc/portage/package.use/installkernel
sys-kernel/installkernel dracut

root # emerge --ask sys-kernel/installkernel
root # emerge --ask sys-kernel/gentoo-kernel-bin
root # emerge --ask sys-kernel/linux-firmware
```

Para qué sirve: es la forma más sencilla de tener un kernel funcional sin configurarlo módulo por módulo. El paquete `gentoo-kernel-bin` instala un kernel ya compilado con parches de Gentoo. `installkernel` con la flag `dracut` genera automáticamente el **initramfs** (un mini sistema que ayuda al kernel a arrancar) cada vez que se instala un kernel nuevo. `linux-firmware` añade el firmware binario que muchas tarjetas de red y GPU necesitan para funcionar.

Afecta: crea `/boot/vmlinuz-*` y `/boot/initramfs-*`, listos para que GRUB los detecte.

⚙ Kernel manual – control total (avanzado)

NO INTERRUMPIR ▼

USO

```
root # emerge --ask sys-kernel/gentoo-sources
root # cd /usr/src/linux
root # make menuconfig
root # make && make modules_install
root # make install
```

Para qué sirve: instala el código fuente del kernel y abre un menú de texto (`make menuconfig`) donde se decide, opción por opción, qué controladores y funciones incluir. Da control total sobre qué se compila, a cambio de requerir conocer bien el hardware del equipo. No se recomienda para el primer intento de instalación.

Afecta: genera un kernel personalizado; un error en la configuración puede resultar en un sistema que no arranca o que no reconoce cierto hardware.

07 Sistema base: locale, zona horaria y red

Ajustes de idioma, hora y conectividad antes del primer arranque.

🕒 ln -sf + eselect locale – zona horaria e idioma

BAJO RIESGO ▼

ZONA HORARIA

```
root # ln -sf ../usr/share/zoneinfo/America/Monterrey /etc/localtime
```

Para qué sirve: `/etc/localtime` es en realidad un enlace simbólico (un "acceso directo") que apunta al archivo de la zona horaria elegida dentro de `/usr/share/zoneinfo/`. `ln -sf` crea ese enlace, reemplazando cualquiera anterior.

LOCALE (IDIOMA Y FORMATO REGIONAL)

```
# /etc/locale.gen
es_MX.UTF-8 UTF-8
en_US.UTF-8 UTF-8

root # locale-gen
root #eselect locale list
root #eselect locale set 4
```

Para qué sirve: el idioma requiere tres pasos. Primero, declarar en `/etc/locale.gen` qué locales generar (cada línea es "idioma_PAÍS.codificación"). Luego correr `locale-gen` para compilarlos de verdad. Por último, `eselect locale set` elige cuál usará el sistema por defecto de la lista que `eselect locale list` mostró.

Afecta: formato de fechas, orden alfabético, mensajes del sistema, y evita advertencias durante compilaciones posteriores (muchos programas esperan al menos un locale UTF-8 disponible).

* netifrc – red con OpenRC

BAJO RIESGO ▼

USO

```
root # emerge --ask --noreplace net-misc/netifrc
```

```
# /etc/conf.d/net – DHCP (dirección automática)
config_eth0="dhcp"

# /etc/conf.d/net – IP estática (dirección fija)
config_eth0="192.168.0.2 netmask 255.255.255.0"
routes_eth0="default via 192.168.0.1"
```

```
root # cd /etc/init.d
root # ln -s net.lo net.eth0
root # rc-update add net.eth0 default
```

Para qué sirve: la interfaz de red se configura declarando variables `config_<interfaz>` en `/etc/conf.d/net`. Con `dhcp` el router asigna la dirección IP automáticamente (opción más simple para la mayoría de los usuarios domésticos). Con una IP estática, tú defines manualmente la dirección exacta y la puerta de enlace.

El enlace `net.eth0` → `net.lo` crea el "script de arranque" de esa interfaz concreta (`netifrc` reutiliza el mismo programa base `net.lo` para cualquier interfaz, solo cambia el nombre del enlace). `rc-update add ... default` agrega ese servicio al runlevel por defecto para que arranque solo junto con el sistema.

COMANDO	EFFECTO
<code>rc-update add <servicio> default</code>	Activa un servicio en el runlevel "default" (arranque normal del sistema).
<code>rc-update del <servicio> default</code>	Lo retira de ese runlevel, para que deje de iniciarse automáticamente.

Afecta: la conectividad de red se activará automáticamente en cada arranque, sin necesidad de configurarla a mano cada vez.

08 fstab

Tabla que le indica al sistema qué partición montar en cada punto y con qué opciones, automáticamente en cada arranque — sin ella, tocaría montar todo a mano cada vez.

≡ /etc/fstab – tabla de montajes

RIESGO MEDIO ▼

EJEMPLO – ESQUEMA UEFI + GPT

# dispositivo	punto	fs	opciones	dump	pass
/dev/sda1	/efi	vfat	umask=0077,tz=UTC	0	2
/dev/sda2	none	swap	sw	0	0
/dev/sda3	/	xfs	defaults,noatime	0	1

EJEMPLO – ESQUEMA MBR + BIOS/LEGACY

# dispositivo	punto	fs	opciones	dump	pass
/dev/sda1	/boot	xfs	defaults	0	2
/dev/sda2	none	swap	sw	0	0
/dev/sda3	/	xfs	defaults,noatime	0	1

Para qué sirve: cada línea define una partición: dispositivo, punto de montaje, tipo de sistema de archivos, opciones de montaje, si se respalda con dump (casi siempre 0, se deja desactivado) y el orden de verificación fsck (1 para la raíz, 2 para el resto, 0 para omitir la verificación).

OPCIÓN	EFFECTO
--------	---------

defaults	Usa un conjunto razonable de opciones estándar (lectura/escritura, montado automático, etc.).
----------	---

noatime	No registra la hora del último acceso a cada archivo; mejora el rendimiento, recomendado especialmente en discos SSD.
---------	---

umask=0077	Restringe los permisos de la ESP a solo root, ya que FAT no soporta permisos de usuario tipo Unix de forma nativa.
------------	--

sw	Marca esa línea como espacio de intercambio (swap), no como una partición de archivos normal.
----	---

Afecta: qué se monta y cómo en cada arranque; un error de sintaxis o un dispositivo mal escrito aquí puede impedir que el sistema arranque con normalidad.

09 Bootloader: GRUB

El paso más delicado de toda la instalación: sin un bootloader correctamente instalado, el firmware no sabe cómo arrancar el kernel.



 grub-install / grub-mkconfig PUNTO CRÍTICO

EMERGE Y ACTIVAR PLATAFORMA EFI

```
root # echo 'GRUB_PLATFORMS="efi-64"' >> /etc/portage/make.conf
root # emerge --ask sys-boot/grub
```

Para qué sirve: esta línea añadida a make.conf le dice a Portage que compile GRUB con soporte para arrancar en modo UEFI de 64 bits. Sin ella, GRUB se compilaría solo con soporte BIOS/legacy.

INSTALAR – ESQUEMA UEFI (CON LA ESP YA MONTADA EN /EFI)

```
root # grub-install --efi-directory=/efi
```

INSTALAR – ESQUEMA BIOS/LEGACY

```
root # grub-install /dev/sda
```

GENERAR EL MENÚ DE ARRANQUE (AMBOS ESQUEMAS)

```
root # grub-mkconfig -o /boot/grub/grub.cfg
```

Para qué sirve: grub-install escribe el programa de GRUB en el lugar correcto para que el firmware lo encuentre al arrancar — en la ESP para UEFI, o en el sector de arranque del disco para BIOS. grub-mkconfig escanea qué kernels hay disponibles en /boot y genera automáticamente el menú de arranque que se ve al encender la máquina.

OPCIÓN	EFFECTO
GRUB_PLATFORMS="efi-64"	Indica a Portage compilar GRUB con soporte UEFI de 64 bits.
--efi-directory=/efi	Ruta donde está montada la ESP; ahí se copian los archivos de arranque de GRUB (solo aplica en UEFI).
/dev/sda (sin número)	En BIOS/legacy, GRUB se instala en el disco completo, no en una partición; por eso no lleva número al final.

--removable

Instala también en la ruta genérica `/EFI/B00T/B00TX64.EFI`, útil si el firmware ignora entradas personalizadas.

`grub-mkconfig -o`
`<ruta>`

Escanea los kernels en `/boot` y guarda (`-o`, "output") el archivo de configuración del menú de arranque en la ruta indicada.

Importante: en el esquema UEFI, la ESP debe estar montada antes de correr `grub-install`. Si no lo está, GRUB puede instalarse en el directorio equivocado sin avisar de forma clara.

Afecta: escribe el cargador de arranque en la ESP (UEFI) o en el disco (BIOS), y genera `/boot/grub/grub.cfg`. Un fallo aquí puede dejar el disco sin forma de arrancar; es el motivo por el que se recomienda no interrumpir este paso ni apagar el equipo hasta confirmar que terminó sin errores.

10

Últimos pasos y primer arranque

Contraseña de root, salida del chroot y reinicio.

 `passwd / exit / reboot`

BAJO RIESGO ▾

USO

```
root # passwd
root # exit
root # cd
root # umount -l /mnt/gentoo/dev{/shm,/pts,}
root # umount -R /mnt/gentoo
root # reboot
```

Para qué sirve: `passwd` establece la contraseña del usuario root dentro del chroot (sin especificar un nombre de usuario, se aplica al usuario actual, que es root). `exit` cierra la sesión de chroot y vuelve al medio en vivo. Los `umount` desmontan de forma ordenada todo lo que se había conectado, para evitar corrupción de datos antes de reiniciar hacia el disco.

OPCIÓN

EFEECTO

`umount -l` Desmontaje "perezoso" (lazy): se desconecta en cuanto deja de estar en uso, sin bloquear si algo sigue accediendo.

`umount -R` Desmonta recursivamente todo lo que esté montado bajo esa ruta, en un solo comando.

Afecta: a partir del reinicio, el sistema arranca desde el disco recién instalado en lugar del medio en vivo (USB/CD).

11 Bifurcación: instalación mínima vs gráfica

A partir de aquí el camino se divide según el objetivo: un servidor/base sin interfaz, o un escritorio listo para usar.

📄 Instalación mínima

stage3 estándar · sin entorno gráfico

- Usa el stage3 `openrc` normal (no el "desktop").
- Ideal para servidores, contenedores, WSL, o para aprender el sistema base sin distracciones.
- Tras el primer arranque solo se tiene una terminal de texto (TTY), sin ratón ni ventanas.
- Para conectividad, `net-misc/dhcpd` o `netifrc` es suficiente.
- Instalar un entorno gráfico queda como paso 100% opcional y posterior, cuando ya se domine lo básico.
- Consumo de disco y tiempos de compilación mucho menores.

🖥️ Instalación gráfica

stage3 "desktop" · entorno gráfico completo

- Se recomienda el stage3 con el sufijo `desktop` : ya trae `llvm` , `rust` y USE flags orientadas a escritorio, acelerando la instalación de programas gráficos después.
- Añadir `USE="X wayland"` o similar en `make.conf` antes de compilar el resto del sistema.
- Definir `VIDEO_CARDS` según la GPU del equipo (ej. `amdgpu radeonsi` , `nvidia` , `i915`).
- Instalar un gestor de pantalla y un entorno de escritorio (ej. `emerge --ask xfce-base/xfce4-meta` o `gnome-base/gnome`).
- Usar `NetworkManager` en vez de `netifrc` si se necesita Wi-Fi con interfaz gráfica de conexión.
- Tiempos de compilación e instalación notablemente mayores — puede tomar varias horas más que la instalación mínima.

Ambos caminos comparten exactamente los mismos pasos 01–10; la bifurcación solo cambia el stage3 elegido, algunas USE flags y el software que se instala después del primer arranque.

12 Comandos esenciales post-instalación

Los comandos con los que convivirás a diario una vez el sistema está arriba.

📦 Gestión de paquetes (Portage)

<code>emerge --sync</code>	Actualiza el árbol de ebuilds (el catálogo de paquetes).
<code>emerge -avuDN @world</code>	Actualiza todo el sistema (pide confirmación, deep, newuse).
<code>emerge -c</code>	Elimina paquetes huérfanos que ya no hacen falta (depclean).
<code>emerge -s <nombre></code>	Busca un paquete por nombre.
<code>equery depgraph <pkg></code>	Muestra el árbol de dependencias (requiere gentoolkit).

⚙️ Servicios (OpenRC)

<code>rc-update add <svc> default</code>	Activa un servicio para que arranque solo con el sistema.
<code>rc-service <svc> start</code>	Inicia un servicio ahora mismo, sin esperar al próximo arranque.
<code>rc-status</code>	Lista qué servicios están activos, agrupados por runlevel.

Usuarios

<code>useradd -m -G wheel,users -s /bin/bash usr</code>	Crea un usuario con carpeta personal (home) y lo añade a grupos.
---	--

<code>passwd usr</code>	Asigna contraseña a ese usuario.
-------------------------	----------------------------------

<code>usermod -aG audio,video usr</code>	Añade grupos adicionales sin quitar los que ya tenía.
--	---

Herramientas de selección

<code>eselect news read</code>	Lee anuncios importantes publicados por el equipo de Portage.
--------------------------------	---

<code>eselect kernel list</code>	Gestiona a qué versión apunta el enlace <code>/usr/src/linux</code> .
----------------------------------	---

<code>env-update && source /etc/profile</code>	Recarga variables de entorno después de cambios de configuración.
--	---